

Basic Information

Experiment No.: 07

Date: 08-10-2026

Title: Java TreeMap and Its Methods

Objective / Aim

To implement "TreeMap" in Java and demonstrate sorted key-value operations, searching, navigation, and retrieving map entries.

System Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM

Software

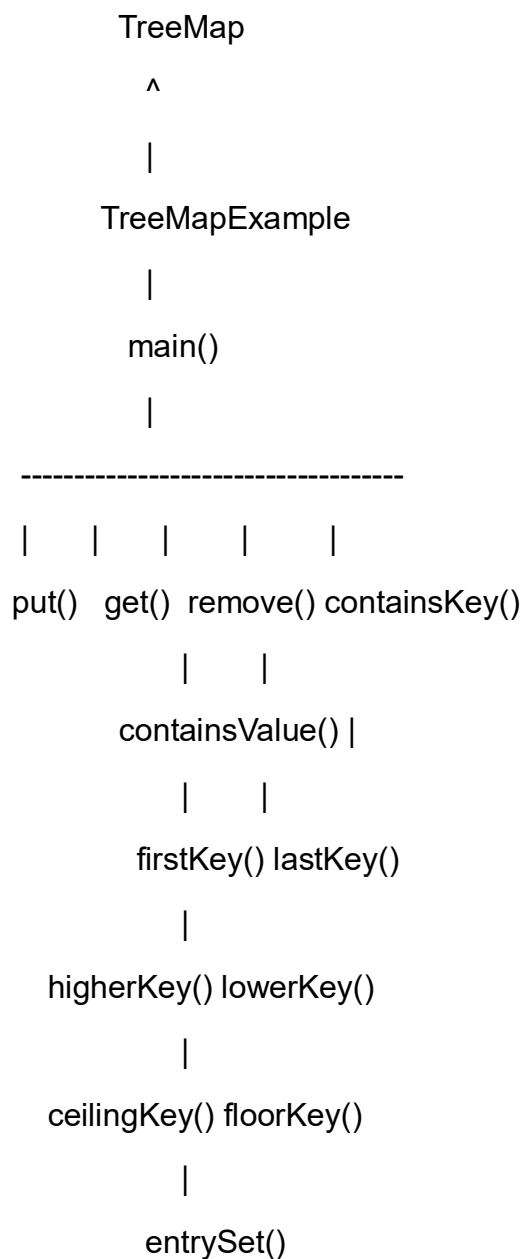
- Java JDK
 - Any Java IDE or Command Prompt
 - Operating System: Windows/Linux/macOS
-

Algorithm / Logic Description

1. Create a "TreeMap" with integer keys and string values.
2. Add key-value pairs using "put()".
3. Access a value using "get()".
4. Check keys and values using "containsKey()" and "containsValue()".
5. Find the first and last keys using "firstKey()" and "lastKey()".

6. Find nearby keys using "higherKey()", "lowerKey()", "ceilingKey()", and "floorKey()".
 7. Display all key-value pairs using "entrySet()".
 8. Remove an entry using "remove()".
 9. End the program.
-

UML Class Diagram



Source Code

```
package collectionframework;

import java.util.TreeMap;

public class TreeMapExample {
    public static void main(String[] args) {

        TreeMap<Integer, String> map = new TreeMap<>();

        // put()
        map.put(103, "John");
        map.put(101, "Rahul");
        map.put(105, "Anil");
        map.put(102, "Priya");
        map.put(104, "Ravi");

        System.out.println("TreeMap: " + map);

        // get()
        System.out.println("Value of key 102: " +
            map.get(102));

        // containsKey()
        System.out.println("Contains key 101: " +
            map.containsKey(101));

        // containsValue()
        System.out.println("Contains value John: " +
```

```
        map.containsValue("John"));

// firstKey()
System.out.println("First Key: " +
    map.firstKey());

// lastKey()
System.out.println("Last Key: " +
    map.lastKey());

// higherKey()
System.out.println("Higher Key than 102: " +
    map.higherKey(102));

// lowerKey()
System.out.println("Lower Key than 102: " +
    map.lowerKey(102));

// ceilingKey()
System.out.println("Ceiling Key of 102: " +
    map.ceilingKey(102));

// floorKey()
System.out.println("Floor Key of 102: " +
    map.floorKey(102));

// entrySet()
System.out.println("Entries: " +
    map.entrySet());
```

```
// remove()
map.remove(105);
System.out.println("After remove: " + map);
}
}
```

Compilation & Execution Commands

Save the file as:

TreeMapExample.java

Since the program contains:

package collectionframework;

the file should be inside the "collectionframework" folder.

Compile:

javac TreeMapExample.java

Run:

java collectionframework.TreeMapExample

Output Screen

TreeMap: {101=Rahul, 102=Priya, 103=John, 104=Ravi, 105=Anil}

Value of key 102: Priya

Contains key 101: true

Contains value John: true

First Key: 101

Last Key: 105

Higher Key than 102: 103

Lower Key than 102: 101

Ceiling Key of 102: 102

Floor Key of 102: 102

Entries: [101=Rahul, 102=Priya, 103=John, 104=Ravi, 105=Anil]

After remove: {101=Rahul, 102=Priya, 103=John, 104=Ravi}

Result

The "TreeMap" class was successfully implemented, and sorted map operations such as "put()", "get()", "remove()", "containsKey()", "containsValue()", "firstKey()", "lastKey()", "higherKey()", "lowerKey()", "ceilingKey()", "floorKey()", and "entrySet()" were successfully demonstrated.